

```

import time
import random
import sys
import pandas as pd
import matplotlib.pyplot as plt

# Implementasi algoritma sorting
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

def selection_sort(arr):
    for i in range(len(arr)):
        min_idx = i
        for j in range(i+1, len(arr)):
            if arr[min_idx] > arr[j]:
                min_idx = j
        arr[i], arr[min_idx] = arr[min_idx], arr[i]

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i-1
        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = key

def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr)//2
        L = arr[:mid]
        R = arr[mid:]
        merge_sort(L)
        merge_sort(R)
        i = j = k = 0
        while i < len(L) and j < len(R):
            if L[i] < R[j]:
                arr[k] = L[i]
                i += 1
            else:
                arr[k] = R[j]
                j += 1
            k += 1
        while i < len(L):
            arr[k] = L[i]
            i += 1
            k += 1
        while j < len(R):
            arr[k] = R[j]
            j += 1
            k += 1

def partition(arr, low, high):
    i = (low-1)
    pivot = arr[high]
    for j in range(low, high):
        if arr[j] <= pivot:
            i = i+1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return (i+1)

def quick_sort_recursive(arr, low, high):
    if len(arr) == 1:
        return arr
    if low < high:
        pi = partition(arr, low, high)
        quick_sort_recursive(arr, low, pi-1)
        quick_sort_recursive(arr, pi+1, high)

def quick_sort(arr):
    quick_sort_recursive(arr, 0, len(arr)-1)

```

```

# Eksekusi dan pengukuran waktu
total_data = [1000, 2000, 3000, 4000, 5000, 6000, 7000, 8000, 9000, 10000]

# Dictionary untuk menyimpan algoritma
algoritma_sorting = {
    "Bubble Sort": bubble_sort,
    "Selection Sort": selection_sort,
    "Insertion Sort": insertion_sort,
    "Merge Sort": merge_sort,
    "Quick Sort": quick_sort
}

# Dictionary untuk menyimpan hasil waktu eksekusi
hasil_waktu = {name: [] for name in algoritma_sorting}

# Loop untuk setiap algoritma dan ukuran data
for data in total_data:
    # Buat data acak dengan angka antara 1 dan ukuran data
    data_asli = [random.randint(1, data) for _ in range(data)]
    print(f"\n--- Menguji dengan {data} elemen data ---")

    for nama_algo, fungsi_sort in algoritma_sorting.items():
        data_untuk_sortir = data_asli[:]

        start_time = time.time()
        fungsi_sort(data_untuk_sortir)
        end_time = time.time()

        waktu_eksekusi = end_time - start_time
        hasil_waktu[nama_algo].append(waktu_eksekusi)
        print(f"{nama_algo}: {waktu_eksekusi:.6f} detik")

# Pembuatan tabel hasil
print("\n" + "="*50)
print("Tabel Hasil Waktu Eksekusi (dalam detik)")
print("="*50)

# Buat DataFrame pandas dari dictionary hasil
df_hasil = pd.DataFrame(hasil_waktu)

# Jadikan total_data sebagai index tabel
df_hasil.index = total_data
df_hasil.index.name = 'Jumlah Data (n)'
pd.options.display.float_format = '{:.6f}'.format

print(df_hasil)
print("\n" + "="*50 + "\n")

df_hasil.to_csv('tabel_waktu_eksekusi.csv')

# Visualisasi hasil komparasi menggunakan grafik
plt.style.use('seaborn-v0_8-darkgrid')
fig, ax = plt.subplots(figsize=(12, 8))

for nama_algo, waktu in hasil_waktu.items():
    ax.plot(total_data, waktu, marker='o', linestyle='-', label=nama_algo)

ax.set_xlabel("Jumlah Elemen Data (n)", fontsize=12)
ax.set_ylabel("Waktu Eksekusi (detik)", fontsize=12)
ax.set_title("Perbandingan Waktu Eksekusi Algoritma Sorting", fontsize=16)
ax.legend(fontsize=12)
ax.tick_params(axis='both', which='major', labelsize=12)
ax.set_yscale('log')
plt.grid(True, which="both", ls="--")
plt.savefig('grafik_perbandingan_sort.png', dpi=300, bbox_inches='tight')

plt.show()

```